

Template-Guided Specification Mining for GR(1)

Sam Kouteili
Yale University
sam.kouteili@yale.edu

Abstract—We present GR1MINE, a SAT-based tool for mining GR(1) specifications from lasso-shaped traces. GR(1) formulas—of the form $(\varphi_I^e \wedge \Box \varphi_S^e \wedge \bigwedge_i \Box \Diamond J_i) \rightarrow (\varphi_I^s \wedge \Box \varphi_S^s \wedge \bigwedge_j \Box \Diamond G_j)$ —are the fragment of LTL for which reactive synthesis is polynomial, yet no prior tool mines them directly from traces. Existing LTL miners search over both temporal and propositional operators, treating GR(1) as just another LTL formula. We exploit the fixed temporal skeleton of GR(1): the temporal structure is given by the template, so only propositional sub-formulas need to be synthesised. This eliminates the temporal operator semantics—the dominant cost in general LTL mining—and decomposes a single large SAT instance into several small, independent ones. On a benchmark suite of 20 GR(1) specifications, GR1MINE solves all 20 while the general LTL miner times out on 4, with speedups ranging from 3.5× to over 15,000×. Moreover, the GR(1) template guarantees that every mined formula is directly usable for reactive synthesis, whereas the baseline frequently produces formulas outside any known realisability fragment.

I. INTRODUCTION

II. PRELIMINARIES

A. Linear Temporal Logic

We consider LTL over a set of atomic propositions $AP = \{x_0, \dots, x_{n-1}\}$. Formulas are built from the grammar

$$\varphi ::= x \mid \neg \varphi \mid \varphi \wedge \varphi \mid \Box \varphi \mid \Diamond \varphi \mid \varphi \mathcal{U} \varphi$$

where $x \in AP$, and the derived operators $\Diamond \varphi \equiv \text{true} \mathcal{U} \varphi$ and $\Box \varphi \equiv \neg \Diamond \neg \varphi$. Semantics are defined over infinite words $w \in (2^{AP})^\omega$ in the standard way [1].

B. Lasso Traces

An *ultimately periodic word* (lasso trace) is a pair (u, v) with $u \in (2^{AP})^*$ and $v \in (2^{AP})^+$, representing the infinite word $u \cdot v^\omega$. We store a lasso trace as a finite vector $\sigma = \sigma[0] \dots \sigma[L-1]$ together with a *lasso start* index ℓ , so that $\sigma[\ell..L-1]$ is the repeating loop. For any position $t < L$, the successor is $t+1$ if $t < L-1$, and ℓ otherwise. We write $\text{loop}(\sigma) = \{\ell, \dots, L-1\}$ for the set of loop positions.

C. GR(1) Specifications

A *Generalised Reactivity(1)* (GR(1)) specification [2] over AP has the form

$$\underbrace{(\varphi_I^e \wedge \Box \varphi_S^e \wedge \bigwedge_{i=1}^m \Box \Diamond J_i)}_{\text{assumptions } A} \rightarrow \underbrace{(\varphi_I^s \wedge \Box \varphi_S^s \wedge \bigwedge_{j=1}^k \Box \Diamond G_j)}_{\text{guarantees } G} \quad (1)$$

where φ_I^e, φ_I^s are propositional *initial conditions*, φ_S^e, φ_S^s are propositional *safety conditions*, and $J_1, \dots, J_m$ and $G_1, \dots, G_k$ are propositional *justice* (liveness) conditions. Every component is a propositional formula over AP (i.e.,

built from $\wedge, \vee, \neg$ and variables only); the temporal operators $\Box, \Diamond$ appear only in the fixed positions shown above.

GR(1) is the largest fragment of LTL for which reactive synthesis is known to be polynomial: the three-nested fixed-point algorithm of Piterman et al. [2] runs in $O(n^3mk)$ on a game graph with n states, compared to the 2EXPTIME-complete general LTL synthesis problem [3].

a) *Semantics on lasso traces.*: A lasso trace σ satisfies $\Box \Diamond J$ iff J holds at some position in $\text{loop}(\sigma)$; it satisfies $\Box \varphi_S$ iff φ_S holds at every position; and it satisfies φ_I iff φ_I holds at position 0. The full formula (1) is an implication: σ satisfies it iff either some assumption fails, or all guarantees hold.

D. The Specification Mining Problem

[Trace separation] Given a set of positive traces P and negative traces N , the *specification mining problem* is to find a formula φ such that $\sigma \models \varphi$ for all $\sigma \in P$ and $\sigma \not\models \varphi$ for all $\sigma \in N$. We seek a smallest such formula in a given fragment.

When the fragment is unrestricted LTL, this is the problem solved by Neider and Gavran [4]. In this work we restrict the fragment to GR(1) and exploit its fixed temporal skeleton.

III. ALGORITHM

IV. EVALUATION

We evaluate GR1MINE against the general LTL miner of Neider and Gavran [4] on a suite of 20 GR(1) benchmarks. All experiments run on a single core of an Apple M-series machine with a 10-minute timeout per benchmark.

A. Benchmark Suite

No standard GR(1) trace-mining benchmarks exist. We construct a suite of 20 specifications covering five dimensions:

- 1) **Variable count** (2–6 atomic propositions).
- 2) **Propositional depth** of the components ($D \in \{1, 2, 3\}$).
- 3) **Template variant**: justice-only (J), with initial conditions (IJ), with safety (SJ), and full GR(1) (ISJ).
- 4) **Multiple justice/guarantee conditions** ($m, k \in \{1, 2\}$).
- 5) **Propositional connectives**: variables only, negation, conjunction, disjunction, and their nesting.

For each benchmark we fix a ground-truth GR(1) formula φ^* and generate traces by random sampling: we draw 5000 random lasso traces of length 4–6 with a random lasso start, evaluate φ^* , and retain 15 positive and 10 negative traces. Both tools receive identical trace sets.

TABLE I

EVALUATION RESULTS. TIMES IN SECONDS; “TO” = 10 MIN TIMEOUT.
 D_e : EXPECTED PROPOSITIONAL DEPTH. Tmpl: TEMPLATE VARIANT.
 SPEEDUP IS THE RATIO OF BASELINE TO GR1MINE WALL-CLOCK TIME.

Benchmark	n	D_e	Tmpl	GR1MINE		Baseline [4]		Speedup
				Time	D	Time	D	
<i>Justice-only, variable scaling</i>								
B01	2	1	J	0.06	1	5.42	5	98×
B02	3	1	J	0.03	1	48.15	6	1 719×
B03	4	1	J	0.03	1	0.60	3	18×
B04	5	1	J	0.04	1	31.93	6	823×
B20	6	1	J	0.04	1	6.32	5	148×
<i>Justice-only, propositional depth</i>								
B05	2	2	J	0.11	2	0.74	3	7×
B06	4	2	J	0.17	2	0.59	3	4×
B07	3	2	J	0.43	3	TO	–	>1 398×
B08	4	2	J	0.15	1	6.63	5	44×
B09	3	3	J	1.07	4	TO	–	>562×
<i>Multiple justice/guarantee conditions</i>								
B10	3	1	J	0.04	1	4.92	5	121×
B11	3	1	J	0.04	1	TO	–	>15 036×
B12	4	1	J	0.09	1	TO	–	>6 771×
<i>Init, safety, and full template</i>								
B13	3	1	IJ	0.05	1	108.86	7	2 191×
B14	3	1	SJ	0.05	1	0.58	3	11×
B15	4	1	ISJ	0.09	1	0.60	3	7×
B16	5	1	ISJ	0.11	1	0.59	3	6×
B17	4	1	ISJ	0.12	1	0.59	3	5×
B18	3	2	ISJ	0.07	1	0.57	3	8×
<i>Complex</i>								
B19	4	2	J	0.65	3	346.73	7	535×

a) Baseline configuration.: The baseline searches over the full LTL operator set $\{\Box, \Diamond, \bigcirc, \mathcal{U}, \wedge, \vee, \neg, \rightarrow\}$ using the DAG encoding of [4], iterating DAG size $D = 1, 2, \dots, 10$. GR1MINE receives the GR(1) template (number of justice/guarantee conditions and whether init/safety slots are active) and iterates $D = 1, 2, \dots$ over the shared propositional depth.

B. Results

Table I reports, for each benchmark, the time to find a separating formula and the DAG depth at which a solution is found.

a) Solving rate.: GR1MINE solves all 20 benchmarks, while the baseline times out on 4 (B07, B09, B11, B12). The timeouts occur on benchmarks with conjunctions in justice conditions (B07, B09) and multiple guarantee conditions (B11, B12)—precisely the cases where the GR(1) structure is hardest to recover from an unconstrained LTL search.

b) Speedup.: On the 16 benchmarks both tools solve, GR1MINE is $3.5\times$ – $2,191\times$ faster (geometric mean: $45\times$). Including the 4 timeouts, lower-bound speedups exceed $15,000\times$. The largest gains arise when the baseline must search deep DAGs ($D \geq 5$) to represent what GR1MINE captures at $D = 1$.

c) DAG depth.: The baseline requires DAG depths 3–7, because it must construct the full temporal structure (e.g., $\Box(\Diamond x_1)$ needs $D \geq 3$). GR1MINE finds most solutions at $D = 1$ since only the propositional content is searched and many specifications use atomic propositions directly.

d) Formula quality.: Beyond speed, the GR(1) template ensures that every mined formula is in the realisable GR(1) fragment and directly usable for synthesis. The baseline, unconstrained, frequently produces formulas outside any known synthesis fragment. Among the 16 benchmarks the baseline solves, we observe three patterns:

- *Meaningful GR(1)-like formulas* (6/16): e.g., $\Box(x_0 \rightarrow \Diamond x_1)$ for benchmark B01. These capture a reactive relationship but are not in canonical GR(1) form.
- *Overly general formulas* (5/16): e.g., $\Box \Diamond x_1$, which satisfies the traces but ignores the assumption side entirely.
- *Spurious separators* (5/16): e.g., $x_1 \mathcal{U} \neg x_1$, which is consistent with the traces but encodes no meaningful reactive specification. This formula holds on any trace where x_1 eventually becomes false, which happens to separate the given traces by coincidence.

By contrast, every formula GR1MINE produces has the canonical GR(1) shape and correctly identifies both the assumption and guarantee components.

V. CONCLUSION

REFERENCES

- [1] A. Pnueli, “The temporal logic of programs,” in *Proc. IEEE Symp. Foundations of Computer Science (FOCS)*, 1977, pp. 46–57.
- [2] N. Piterman, A. Pnueli, and Y. Sa’ar, “Synthesis of reactive(1) designs,” in *Proc. Verification, Model Checking, and Abstract Interpretation (VMCAI)*, 2006, pp. 364–380.
- [3] A. Pnueli and R. Rosner, “On the synthesis of a reactive module,” in *Proc. ACM Symp. Principles of Programming Languages (POPL)*, 1989, pp. 179–190.
- [4] D. Neider and I. Gavran, “Learning linear temporal properties,” in *Proc. Formal Methods in Computer-Aided Design (FMCAD)*, 2018, pp. 1–10.